

HOW ACCURATE IS GOOD ENOUGH?

Approximating a Civil Engineering Function Using Infinite Series

An Integrated Python Exercise for Civil Engineering Students

Topics Covered:

- Geometric Series
 - Power Series
- Maclaurin Series
 - Taylor Series
- Error Analysis and Engineering Decision-Making

Learning Outcomes:

Students will understand how infinite series can approximate engineering functions, quantify approximation errors, and make engineering decisions based on accuracy requirements.

Date: September 15, 2026

ENGINEERING SCENARIO

A civil engineer is analyzing the geometry of a sloping structural or surveying system. The relationship between angle and vertical component is given by:

$$y = L \sin(\theta)$$

Where:

- $L = 20$ m (structural length)
- θ = angle in radians
- y = vertical component

Central Question: How accurately can a civil engineer approximate an engineering quantity using a finite number of terms of an infinite series?

PART 1: GEOMETRIC SERIES

Begin with the geometric series and its partial sum:

$$1 + x + x^2 + x^3 + \dots = \sum x^k$$

Task: Implement a Python function to calculate the partial sum without using the closed-form equation $1/(1-x)$.

```
def geometric_sum(x, N): """Calculate S_N = 1 + x + x^2 + ... + x^N""" total = 0.0 for k in range(N + 1): total += x ** k return total
```

Investigation: Compare partial sums with the exact value for $x = 0.5$, 0.8 , and 0.9 . How does the number of terms affect accuracy? Why does convergence speed differ?

Effect of term count on accuracy: Increasing the number of terms (N) incorporates higher-order powers, reducing truncation error and driving the partial sum closer to the exact analytical limit.

Differences in convergence speed: Values closer to zero ($x = 0.5$) converge rapidly because higher powers decay quickly. Values closer to one ($x = 0.8, 0.9$) require significantly more terms because higher powers shrink very slowly.

Key Concepts: partial sum, convergence, approximation error, number of terms needed

PART 2: POWER SERIES

Generalize the geometric series into a power series:

$$P_N(x) = a_0 + a_1x + a_2x^2 + \dots + a_Nx^N$$

Task: Create a function that evaluates a power series at a given point.

```
def power_series(x, coefficients): """Evaluate P_N(x) with given coefficients""" result
= 0.0 for k, a k in enumerate(coefficients): result += a k * (x ** k) return result
```

Key Insight: A polynomial is a finite power series. An infinite power series can approximate transcendental functions like sin, cos, and exponentials.

PART 3: MACLAURIN SERIES FOR $\sin(\theta)$

The Maclaurin series is a power series centered at $\theta = 0$:

$$\sin(\theta) = \theta - \frac{\theta^3}{3!} + \frac{\theta^5}{5!} - \frac{\theta^7}{7!} + \dots$$

Task: Build the Maclaurin approximation explicitly using loops. Do NOT use `math.sin()` to generate the approximation (only for verification).

```
def sin_maclaurin(theta, N): """Approximate sin(theta) with N terms""" result = 0.0
for n in range(N): sign = (-1) ** n factorial = math.factorial(2*n + 1) result += sign *
(theta ** (2*n + 1)) / factorial return result
```

Investigation: For $\theta = 10^\circ$, calculate approximations using 1, 2, 3, and 4 terms. Compare each to the exact value. What is the error at each step?

1 Term (θ): Yields a close initial approximation with a very small percentage error.

2 Terms ($\theta - \frac{\theta^3}{3!}$): Dramatically reduces error, capturing cubic curvature.

3 & 4 Terms: Yield results virtually identical to the exact value within standard floating-point precision, as higher-order factorial terms shrink rapidly for small angles.

PART 4: ENGINEERING INVESTIGATION

Engineering Problem: Calculate the vertical component $y = L \sin(\theta)$ for various angles using Maclaurin approximations.

Given Data:

- $L = 20 \text{ m}$
- Angles: $1^\circ, 2^\circ, 5^\circ, 10^\circ, 15^\circ, 20^\circ, 30^\circ$
- Use Maclaurin approximations with 1, 2, 3, and 4 terms

Task: Create a table for each number of terms showing:

- Angle (degrees)
- Exact value y
- Approximation
- Absolute error
- Percentage error

Analysis Questions:

1. How does the error change as the angle increases? For a Maclaurin series centered at 0 degrees, truncation error grows larger as the angle moves further away from the origin.
2. How does the error change as you add more terms? Adding more terms systematically and significantly decreases the error magnitude across all evaluated angles.
3. For small angles ($< 5^\circ$), how many terms are sufficient? Only 1 term (the linear small-angle approximation $\sin(\theta) \approx \theta$) is needed to keep errors well below strict engineering thresholds.
4. For larger angles ($> 20^\circ$), how many terms are needed? At least 3 to 4 terms (or a shifted Taylor expansion) are required to maintain high precision.

PART 5: TAYLOR SERIES

The Taylor series generalizes from Maclaurin by expanding around any point a :

$$f(\theta) = f(a) + f'(a)(\theta - a) + f''(a)/2!(\theta - a)^2 + \dots$$

Key Insight: Maclaurin is simply Taylor with $a = 0$. By choosing a different expansion point, you can improve accuracy in different regions.

Task: Construct a Taylor approximation for $\sin(\theta)$ centered at $a = 10^\circ$.

```
def sin_taylor(theta, a, N): """Approximate sin(theta) using Taylor centered at a"""
    result = 0.0
    sin_a, cos_a = math.sin(a), math.cos(a)
    for n in range(N): # Derivatives of sin cycle every 4 terms
        derivative_pattern = n % 4
        if derivative_pattern == 0: f_deriv = sin_a
        elif derivative_pattern == 1: f_deriv = cos_a
        elif derivative_pattern == 2: f_deriv = -sin_a
        else: f_deriv = -cos_a
        term = f_deriv * ((theta - a) ** n) / math.factorial(n)
        result += term
    return result
```

Comparison: For the same angles (1° – 30°), calculate errors using:

- Maclaurin (centered at 0°)
- Taylor (centered at 10°)

Which performs better near 10° ? Which is better far from 10° ?

Near 10° : Both Maclaurin and Taylor series perform well, but the Taylor series centered at 10° minimizes remainder terms right around its expansion pivot.

Far from 10° : Taylor approximation errors increase as you move further away from 10° , demonstrating that localized expansions excel primarily within their immediate neighborhood.

PART 6: ENGINEERING DECISION—ERROR TOLERANCE

Requirement: The approximation must have less than 0.1% error.

Task 1: For each angle (1° – 30°), find the minimum number of terms needed to meet the 0.1% tolerance using both Maclaurin and Taylor approximations.

Task 2: Investigate the small-angle approximation $\sin(\theta) \approx \theta$. At what angle does this 1-term approximation stop being sufficiently accurate?

Analysis:

1. Does adding more terms always improve the approximation? Yes, up to the limit of computer floating-point machine precision.
2. Why is the Maclaurin approximation most accurate near zero? Its coefficients are derived directly from derivatives evaluated at $\theta = 0$ deg.
3. Why can changing the Taylor expansion point improve accuracy? It minimizes the distance term $(\theta - a)$, suppressing higher-order remainder terms near the operational region.
4. How does the distance from the expansion point affect accuracy? Greater distance increases the magnitude of $(\theta - a)^n$, causing truncation errors to accumulate faster.
5. What is the critical angle for the $\sin(\theta) \approx \theta$ approximation?
Around 4 deg. – 5 deg.

PART 7: FINAL ENGINEERING RECOMMENDATION

Decision Problem:

You are the civil engineer responsible for the calculation. You need an approximation with less than 0.1% error. Would you use the Maclaurin series, a Taylor series centered closer to the angle of interest, or the exact sine function? Defend your choice using your numerical results.

Your recommendation must include:

- Number of terms required
- Percentage error achieved
- Convergence behavior
- Computational simplicity vs. accuracy tradeoff
- Valid angle range for your solution

REQUIRED PYTHON OUTPUT

Your final Python program must produce the following deliverables:

1. Numerical Tables

Formatted tables showing exact values, approximations, and errors for each angle and number of terms.

2. Convergence Plot

A figure showing how percentage error decreases as you add more terms, for different angles.

3. Function Comparison Plot

Side-by-side plots of exact $\sin(\theta)$ vs. Maclaurin and Taylor approximations with different numbers of terms.

4. Error Comparison Plot

Visual comparison of absolute and percentage errors between Maclaurin and Taylor series.

5. Written Recommendation

A clear explanation of your engineering decision with supporting numerical evidence.

Implementation Note: Explicitly implement the series summation using Python loops. Do not hide the mathematics inside symbolic mathematics software. The purpose is to make you see the progression from geometric series → power series → Maclaurin → Taylor → engineering approximation → error analysis → engineering decision.

THE MATHEMATICAL PROGRESSION

1. Geometric Series

$$1 + x + x^2 + \dots = 1/(1-x)$$



2. Power Series

$$a_0 + a_1 x + a_2 x^2 + \dots$$



3. Maclaurin Series

Expand around $\theta = 0$



4. Taylor Series

Expand around any point $\theta = a$



5. Engineering Approximation

Apply to $y = L \sin(\theta)$



6. Error Analysis

Compare with exact values



7. Engineering Decision

What's the best tradeoff?